

Course Outline: Object-Oriented Programming in Python

Title: Object-Oriented Programming in Python: From Blueprints to Reality **Learnpack:** + Programación Orientada a Objetos
Prerequisite: Python fundamentals, functions, data structures **Target Audience:** Beginner developers with Python basics who need to understand OOP for building scalable applications and reading existing codebases **Total Lessons:** 11 **Format:** Mixed (27% hands-on)

Course Description

This course introduces object-oriented programming (OOP) concepts through Python, teaching you to model real-world problems using classes and objects. You'll learn the fundamental difference between a blueprint (class) and its instances (objects), then master the four core principles that make OOP powerful: abstraction, encapsulation, inheritance, and polymorphism.

By the end, you'll understand how to create reusable, maintainable code by organizing data and behavior into cohesive units. This knowledge is essential for building scalable applications, working with databases, and understanding most modern codebases you'll encounter as a developer.

The course emphasizes practical application over theory, showing you how one class can create hundreds of unique objects, each with their own state and behavior, and how this approach solves real programming challenges you'll face in web development and beyond.

Course Philosophy

- This course emphasizes:
- **State + Behavior Pattern:** Objects should contain both data (attributes) and the methods that act on that data
 - **Model-first thinking:** Understanding how real-world entities translate into code structures
 - **Practical application:** Focus on how OOP solves actual problems in web development and database modeling
 - **Clean boundaries:** Learning when to use OOP vs when simpler approaches work better

Course Statistics Summary

Section	Lessons
00 - Welcome	1
01 - Classes and Objects Fundamentals	3
02 - Object Behavior and State	2
03 - OOP Core Principles	3
04 - Assessment and Integration	2
05 - Next Steps	1
Total	11

Section 00: Welcome

00.0 Welcome to Object-Oriented Programming

Learning Objectives:

- Understand why OOP exists and what problems it solves in real applications
- Recognize the connection between OOP and database modeling from previous lessons
- Identify when OOP is the right tool vs simpler programming approaches

Content Outline:

- The core problem: How do you organize complex applications with hundreds of users, products, and orders?
- From individual variables to grouped data and behavior
- Preview: One Enemy class creating 100 unique orcs, each with their own health and position
- Connection to database models and web development

Transitions: Next, we'll explore what a class actually is and how it serves as a blueprint for creating multiple similar objects.

Key Principles:

- **Organization over complexity:** OOP helps manage large applications by grouping related data and behavior
- **Reusability:** Write once, create many instances with different states
- **Real-world modeling:** Code structures that mirror how we think about entities and their relationships

Examples:

- A User class creating multiple user accounts, each with unique usernames and email addresses
 - A Product class generating hundreds of items in an e-commerce system, each with different prices and inventory levels
-

Section 01: Classes and Objects Fundamentals

01.0 What is a Class? Understanding the Blueprint 📖

Learning Objectives:

- Define what a class is and how it serves as a template
- Identify the components that make up a class structure
- Understand the relationship between classes and the real-world entities they model

Content Outline:

- Class as blueprint: The architectural plan vs the actual building
- Anatomy of a Python class: class keyword, naming conventions, basic structure
- Attributes vs methods: data storage vs behavior
- The `__init__` constructor: setting up new instances with initial values

Transitions: Now that we understand what classes are, let's explore the crucial difference between the blueprint itself and the actual objects it creates.

Key Principles:

- **Template pattern:** Classes define structure and behavior that instances will have
- **Initialization:** Every object needs a consistent setup process through `__init__`
- **Naming convention:** Classes use PascalCase (UserProfile) vs functions using snake_case

Examples:

- A Car class defining attributes like color, model, year and methods like `start_engine()`
- A BankAccount class with balance attribute and deposit/withdraw methods

01.1 Classes vs Objects: From Blueprint to Reality 📖

Learning Objectives:

- Distinguish clearly between class definitions and object instances
- Understand how multiple objects can be created from the same class
- Recognize that each object maintains its own independent state

Content Outline:

- The fundamental distinction: class vs instance
- Creating objects: instantiation process and memory allocation
- Independent state: `orco1.vida = 100`, `orco2.vida = 50` - same class, different data
- Multiple instances in action: `user1`, `user2`, `user3` all from `User` class

Transitions: Understanding the theory is essential, but let's now get hands-on experience creating classes and instantiating multiple objects from them.

Key Principles:

- **Instance independence:** Each object maintains its own attribute values
- **Shared behavior:** All instances can use the same methods defined in the class
- **Memory efficiency:** Methods exist once in the class, but each instance has its own data
- **The self parameter:** How methods know which specific object they're acting upon

Examples:

- Enemy class creating `orco1`, `orco2`, `orco3` - each with different health values
- Student class generating multiple student records with unique names and grades

01.2 Creating Your First Classes and Objects

Challenge Prompt: Create a `Videogame` class that can generate multiple game instances, each with a title, genre, and rating, then create three different game objects and display their information.

Solution Code:

```
class Videogame:
    def __init__(self, title, genre, rating):
        self.title = title
        self.genre = genre
        self.rating = rating

    def display_info(self):
        return f"{self.title} - {self.genre} (Rating: {self.rating}/10)"

# Create three game instances
game1 = Videogame("The Legend of Zelda", "Adventure", 9)
game2 = Videogame("Call of Duty", "FPS", 8)
game3 = Videogame("SimCity", "Simulation", 7)

# Display each game's information
print(game1.display_info())
print(game2.display_info())
print(game3.display_info())
```

Challenge Code:

```
class Videogame:
    def __init__(self, title, genre, rating):
        # Initialize the game attributes
```

```
# Your code here
pass

def display_info(self):
    # Return formatted string with game information
    # Your code here
    pass

# Create three different game instances
# Your code here

# Display information for each game
# Your code here
```

Section 02: Object Behavior and State

02.0 Methods and Attributes: State + Behavior Pattern

Learning Objectives:

- Understand the State + Behavior pattern as the foundation of OOP design
- Learn how methods should act on object attributes to maintain data integrity
- Recognize the `self` parameter as the way methods access the specific object's data

Content Outline:

- State (attributes) vs Behavior (methods): data and the actions that modify it
- The self parameter: not magic, just the first argument referring to the current instance
- Instance attributes vs class attributes: when data belongs to the object vs the class
- Method design: functions that work with the object's internal state

Transitions: With a solid understanding of how state and behavior work together, let's practice building interactive objects that can respond to method calls and maintain their state over time.

Key Principles:

- **State + Behavior:** Objects should contain both data and the methods that act on that data
- **Self reference:** Methods need `self` to know which specific object they're operating on
- **Data integrity:** Methods should be the primary way to modify object attributes
- **Instance attributes:** Always define attributes inside `__init__` for clear object initialization

Examples:

- A Player class with health attribute and `take_damage()` method that reduces health
- A BankAccount class with balance attribute and `deposit()/withdraw()` methods that modify the balance

Anti-patterns Integration:

- **Static Class Anti-pattern:** Avoid classes that are just containers for functions (like a Math class with only static methods) - that's not OOP, it's just function grouping
- **External Manipulation:** Don't modify attributes directly from outside (`player.xp += 100`) - use methods instead (`player.gain_experience(100)`) for better control and validation

02.1 Building Interactive Objects

Challenge Prompt: Create a `Character` class with name, health, and mana attributes, plus methods to `cast_spell` (costs mana, deals damage to target) and `rest` (restores mana), then simulate a character casting spells and resting.

Solution Code:

```
class Character:
    def __init__(self, name, health=100, mana=50):
        self.name = name
        self.health = health
        self.mana = mana

    def cast_spell(self, spell_cost=10):
        if self.mana >= spell_cost:
            self.mana -= spell_cost
            return f"{self.name} casts spell! Mana: {self.mana}"
        return f"{self.name} doesn't have enough mana!"

    def rest(self, mana_restored=20):
        self.mana += mana_restored
        return f"{self.name} rests. Mana: {self.mana}"

# Create character and interact
wizard = Character("Gandalf", health=120, mana=80)
print(wizard.cast_spell(15))
print(wizard.cast_spell(70))
print(wizard.rest(30))
print(wizard.cast_spell(25))
```

Challenge Code:

```
class Character:
    def __init__(self, name, health=100, mana=50):
        # Initialize character attributes
        # Your code here
        pass

    def cast_spell(self, spell_cost=10):
        # Check if enough mana, reduce mana, return message
        # Your code here
        pass

    def rest(self, mana_restored=20):
        # Restore mana and return message
        # Your code here
        pass

# Create a character and test the methods
# Your code here
```

Section 03: OOP Core Principles

03.0 Abstraction and Encapsulation: Hiding Complexity 📖

Learning Objectives:

- Understand abstraction as hiding implementation details while exposing essential functionality
- Learn encapsulation as controlling access to object data through methods
- Recognize how these principles improve code maintenance and reliability

Content Outline:

- Abstraction: providing simple interfaces for complex operations (`car.start()` vs managing fuel injection, spark plugs, etc.)
- Encapsulation: controlling data access through methods rather than direct attribute manipulation
- Private attributes convention: using underscore prefix (`_attribute`) to signal internal use
- Public interface design: which methods should be available to users of your class

Transitions: These principles of hiding complexity lead us to inheritance, where we can share common functionality across related classes while maintaining clean boundaries.

Key Principles:

- **Interface clarity:** Users should only see methods they need to accomplish their goals
- **Data protection:** Control how object data can be modified to prevent invalid states
- **Implementation hiding:** Internal complexity should be hidden behind simple method calls
- **Consistent access:** Use methods rather than direct attribute access for better control

Examples:

- A `DatabaseConnection` class with simple `connect()` method hiding all the connection string, authentication, and error handling complexity
- A `ShoppingCart` class with `add_item()` method that handles inventory checking, price calculation, and tax computation internally

Anti-patterns Integration:

- **Total Access Anti-pattern:** Avoid letting external code directly modify all attributes (`player.xp += 100`) - if you later need level limits, you'll have to find and change code everywhere
- **External Functions Anti-pattern:** Don't create functions outside the class that manipulate the object - move that behavior into the class as methods

03.1 Inheritance: Extending and Reusing Code

Learning Objectives:

- Understand inheritance as a way to share common functionality between related classes
- Learn to create parent classes with shared attributes and methods
- Master the `super()` function for extending parent class behavior

Content Outline:

- Parent-child relationships: base classes and derived classes
- Inheriting attributes and methods: what gets passed down automatically
- Method overriding: customizing behavior in child classes
- The `super()` function: extending rather than replacing parent functionality
- When to use inheritance vs composition

Transitions: Inheritance gives us code reuse, but polymorphism takes this further by letting us treat different objects uniformly while they behave according to their specific types.

Key Principles:

- **Code reuse:** Share common functionality through inheritance rather than copying code
- **Specialization:** Child classes add specific behavior while maintaining parent capabilities
- **Method resolution:** Python looks for methods starting from the child class up to parents
- **Super extension:** Use `super()` to build upon parent functionality rather than replacing it

Examples:

- A `Vehicle` parent class with `start_engine()` method, inherited by `Car` and `Motorcycle` child classes
- An `Employee` parent class with `calculate_pay()` method, extended by `SalariedEmployee` and `HourlyEmployee`

Anti-patterns Integration:

- **Deep Hierarchy Anti-pattern:** Avoid inheritance chains like Animal -> Mammal -> Canine -> Dog -> Poodle - they become impossible to maintain and understand

03.2 Polymorphism: Many Forms, Same Interface

Learning Objectives:

- Understand polymorphism as the ability to treat different objects uniformly
- Learn how method overriding enables different behaviors with the same method names
- Recognize polymorphism's power in creating flexible, extensible code

Content Outline:

- Same interface, different behavior: calling the same method name on different object types
- Method overriding in practice: how child classes customize inherited methods
- Polymorphism in action: lists of different object types responding to the same method calls
- Duck typing in Python: "if it walks like a duck and quacks like a duck, it's a duck"

Transitions: Now that we understand all four OOP principles, let's put them together in a comprehensive coding challenge that demonstrates their combined power.

Key Principles:

- **Uniform interface:** Different objects can respond to the same method calls in their own way
- **Runtime behavior:** The actual method executed depends on the object type, determined at runtime
- **Extensibility:** New classes can be added without changing existing code that uses the interface
- **Code flexibility:** Write code that works with any object implementing the expected interface

Examples:

- A list of different Shape objects (Circle, Square, Triangle) all responding to calculate_area() with their specific formulas
- Different PaymentProcessor objects (CreditCard, PayPal, BankTransfer) all implementing process_payment() differently

Best Practices Integration:

- **Objects within Objects Pattern:** Show how a Car class can have a list of Wheel objects, each with their own tire pressure and wear level
- **List of Objects Pattern:** Demonstrate a single users = [] list where each element is a User instance with unique data

Section 04: Assessment and Integration

04.0 Object-Oriented Design Patterns

Challenge Prompt: Design a simple library system with Book and Library classes where the Library can add books, find books by title, and list all available books, demonstrating inheritance, encapsulation, and the State + Behavior pattern.

Solution Code:

```
class Book:
    def __init__(self, title, author, isbn):
        self.title = title
        self.author = author
        self.isbn = isbn
        self.is_available = True

    def checkout(self):
        if self.is_available:
```

```

        self.is_available = False
        return True
    return False

def return_book(self):
    self.is_available = True

class Library:
    def __init__(self, name):
        self.name = name
        self.books = []

    def add_book(self, book):
        self.books.append(book)
        return f"Added '{book.title}' to {self.name}"

    def find_book(self, title):
        for book in self.books:
            if book.title.lower() == title.lower():
                return book
        return None

    def list_available_books(self):
        available = [book.title for book in self.books if book.is_available]
        return available

# Test the library system
library = Library("City Library")
book1 = Book("1984", "George Orwell", "123456")
book2 = Book("To Kill a Mockingbird", "Harper Lee", "789012")

print(library.add_book(book1))
print(library.add_book(book2))
print("Available books:", library.list_available_books())

```

Challenge Code:

```

class Book:
    def __init__(self, title, author, isbn):
        # Initialize book attributes including availability
        # Your code here
        pass

    def checkout(self):
        # Change availability and return success/failure
        # Your code here
        pass

    def return_book(self):
        # Make book available again
        # Your code here
        pass

class Library:
    def __init__(self, name):
        # Initialize library with name and empty book list
        # Your code here
        pass

```



```

def add_book(self, book):
    # Add book to library and return confirmation message
    # Your code here
    pass

def find_book(self, title):
    # Search for book by title, return book or None
    # Your code here
    pass

def list_available_books(self):
    # Return list of available book titles
    # Your code here
    pass

# Create library and books, test the functionality
# Your code here

```

04.1 OOP Mastery Assessment

Learning Objectives:

- Evaluate understanding of classes vs objects distinction
- Assess knowledge of the four OOP principles and their applications
- Test ability to identify appropriate use cases for OOP patterns
- Verify comprehension of best practices and anti-patterns

Question Format: Multiple choice

Assessment Questions:

1. What is the main difference between a class and an object?

- A) Classes store data, objects store methods
- B) A class is a blueprint/template, an object is a specific instance created from that class
- C) Classes are written in Python, objects are written in JavaScript
- D) Objects are more complex than classes

2. In the code `player1 = Player("John", 100)` and `player2 = Player("Jane", 100)` , what is true about player1 and player2?

- A) They are the same object with different names
- B) They share the same memory location
- C) They are separate objects that can have independent attribute values
- D) Changing player1.health will automatically change player2.health

3. What does the `self` parameter represent in a Python class method?

- A) It's a magic keyword that Python requires
- B) It refers to the specific object instance the method is being called on
- C) It represents all objects of that class
- D) It's used to create new objects

4. Which OOP principle is demonstrated when a `Car` class has an `engine` attribute that is itself an `Engine` object?

- A) Inheritance - Car inherits from Engine
- B) Polymorphism - Different cars behave differently
- C) Composition - Objects within objects pattern
- D) Abstraction - Hiding engine complexity

5. What is wrong with this approach: `user.balance += 100` instead of `user.deposit(100)` ?

- A) It's more efficient to modify attributes directly
- B) It violates encapsulation by bypassing method-based control and validation
- C) Python doesn't allow direct attribute access
- D) It will cause a syntax error

6. In inheritance, what does `super()` allow you to do?

- A) Create a new parent class
- B) Delete the parent class functionality
- C) Call and extend the parent class method rather than completely replacing it
- D) Make the child class more powerful than the parent

7. Which scenario best demonstrates polymorphism?

- A) Creating multiple objects from the same class
- B) A list containing Circle, Square, and Triangle objects all responding to `calculate_area()` with their specific formulas
- C) Inheriting attributes from a parent class
- D) Hiding implementation details behind method calls

Evaluation Criteria:

- 7/7 correct: Full mastery of OOP concepts
- 5-6 correct: Good understanding with minor gaps
- 3-4 correct: Basic comprehension, review core principles
- 0-2 correct: Need to revisit fundamental concepts

Score Interpretation: This assessment evaluates your grasp of object-oriented programming foundations essential for building scalable applications and working with modern frameworks that heavily use OOP patterns.

Section 05: Next Steps

05.0 From Objects to Applications

Content Outline:

- Course recap: You've learned to model real-world entities as classes, create multiple independent objects, and organize code using the four OOP principles
- Connection to bigger picture: These concepts form the foundation for database models, web frameworks like Django/FastAPI, and most modern application architectures
- Immediate application: You can now understand and work with existing codebases that use OOP patterns, and start designing your own applications with proper object structure
- Next learning steps: Advanced OOP topics (decorators, property methods), design patterns (Observer, Factory, Strategy), and OOP in web frameworks
- Resources for continued learning: Python documentation on classes, "Clean Code" principles for object design, framework-specific OOP patterns
- Encouragement: You've mastered fundamental concepts that separate beginner programmers from those ready to build professional applications

Note: This is conclusion only - no theory, exercises, or assessment.
